

Ion energy and angular distributions from a SOMAFOAM run

Running the cross sections → fluid → ion tracking chain for other conditions

1 What the chain does

Three programs are run one after the other; each uses the output of the one before.

Step	Program	Input → output
1	<code>electronBoltzmann</code>	electron cross sections → electron mobility, diffusion coefficient and rate coefficients against the electron temperature
2	<code>somaFoam</code>	those tables, the ion mobility and the discharge conditions → densities, electron temperature and the electric field over the RF period
3	<code>ionTracker</code>	the electric field at a number of phases of one period and ion–neutral cross sections → energy and angular distributions of the ions at the electrodes

The worked case is `examples/plasma/HeliumIonDistribution`: a one-dimensional helium discharge, 6.7 cm gap, $3.21 \times 10^{22} \text{ m}^{-3}$ (133 Pa) at 300 K, 120 V at 13.56 MHz, absorbing electrodes without secondary emission. These are the conditions of case 4 of Turner *et al.*, Phys. Plasmas **20**, 013507 (2013), a benchmark for particle-in-cell codes. After `source <SOMAFOAM>/etc/bashrc`, `./Allrun` runs all the steps (about seven hours on one core, nearly all in step 2) and `./caseClean` removes the results. Sections 3 to 5 follow that script, section 6 says what to change for other conditions, and section 8 gives the results of the example.

2 Files of the case

File	Contents
<code>constant/He_electron.txt</code>	electron–neutral cross sections, LXCat format
<code>constant/He+_He.txt</code>	ion–neutral cross sections, LXCat format (blocks ISOTROPIC and BACKSCAT)
<code>constant/mu_Hep1</code>	ion mobility: E/N [Td] and μN [1/(V m s)]
<code>system/boltzmannDict</code>	step 1: gas, range of E/N , tables to write
<code>constant/plasmaProperties</code>	species, transport models, flux scheme; <code>electronBoltzmann yes</code> ; makes <code>somaFoam</code> run step 1 itself
<code>constant/reactions</code>	reactions, the table of each and its energy loss [J]
<code>constant/thermoProperties</code>	species and their masses
<code>constant/polyMesh/blockMeshDict</code>	gap and number of cells
<code>0/He, 0/p, 0/T</code>	gas density [1/m ³], pressure [Pa], temperature [K]
<code>0/Phi</code>	voltage amplitude and frequency of the driven electrode
<code>0/electron, 0/Hep1, 0/Te</code>	initial values and wall conditions (secondary emission coefficient <code>seec</code>)
<code>system/setExpressionFieldsDict</code>	initial density profile
<code>system/controlDict</code>	time step, run length, output
<code>system/ionTrackerDict</code>	step 3
<code>Allrun, caseClean</code>	run all the steps; remove the results
<code>warpx/inputs</code>	input of the particle-in-cell code WarpX for the same case (section 8)

Written by step 1:

<code>constant/mu_electron, D_electron</code>	electron mobility and diffusion coefficient times the gas density, against T_e [K]
<code>constant/reaction_elastic, _exc1, _exc2, _ion</code>	rate coefficients against T_e [K]
<code>constant/boltzmann/</code>	all the results against E/N , and the energy distributions

3 Step 1: electron tables

```
file      "constant/He_electron.txt";
gases     ((He 1.0));           // (target fraction) ...
gasTemperature 300;
reducedField { min 0.001; max 300; n 111; } // Td
nEnergyCells 600;
tables
{
    mobility  "constant/mu_electron";
    diffusion "constant/D_electron";
    rates     (("constant/reaction_elastic" 0) ("constant/reaction_exc1" 1)
              ("constant/reaction_exc2" 2) ("constant/reaction_ion" 3));
}
```

The numbers in `rates` are the positions of the processes in the list that `electronBoltzmann` prints (the order of the cross-section file). The file names are those used in `constant/reactions`.

- Run it by hand with `electronBoltzmann`, or leave `electronBoltzmann yes;` in `constant/plasmaProperties`.
- The tables are against the electron temperature $T_e = \frac{2}{3}\langle\epsilon\rangle/k_B$ and do not depend on the pressure: they need to be made again only if the gas, the mixture or the gas temperature changes.
- The range of E/N must give electron temperatures that cover those of the discharge. `constant/boltzmann/transport.dat` lists T_e for every E/N . Outside the table the fluid solver uses the end value.
- `method monteCarlo;` replaces the two-term solution by a Monte Carlo swarm simulation. Use it to check the two-term tables where ionisation is strong (above roughly 100 Td in argon). In helium the two-term solution has no steady state above a few hundred Td; the example stops at 300 Td.

4 Step 2: fluid solution

```
blockMesh
setExpressionFields
somaFoam
```

Run length. The densities settle on the ambipolar diffusion time, which is thousands of RF periods at this pressure. The example starts from a cosine profile near the expected density (`setExpressionFieldsDict`) and runs 6000 periods of 250 time steps; the mid-plane density is steady to 0.1% after about 4000. Check that the run has settled before step 3:

```
<SOMAFOAM>/postprocessing/plot_1d.py . --fields N_Hep1Mean TeMean --history
```

Phase-resolved output. `ionTracker` needs the fields `E`, `J_<ion>`, the ion and electron densities, the gas density and `T` at `nPhases` equally spaced times of one period. `Allrun` continues the run for one period with

```
startFrom latestTime;
endTime   <latest time + one period>;
writeControl  timeStep;
writeInterval 5; // 250 steps per period / 5 = 50 phases
```

The number of time steps per period must be a multiple of `nPhases`. `E` and `J_<ion>` are written because they are listed in `objectNames` of the `dumpObjects` function in `system/controlDict`.

5 Step 3: ion distributions

```

ion            Hep1;
ionMass        4.002602;           // amu
charge         1;
electricField  E;
gasDensity     He;
gasTemperature T;
startTime      4.4247788e-04;      // first of the nPhases outputs (set by Allrun)
period         7.3746312684e-08;    // 1/13.56 MHz
nPhases        50;
source         fluxDivergence;      // needs J_Hep1 in the outputs
launch         sheath;              // track the sheaths only
nParticles     100000;
nThreads       4;
patches        (electrode ground);
crossSections  LXCat;
file           "constant/He+_He.txt";
energyFrame    centreOfMass;        // frame of the energies of the file
maxCollisionEnergy 300;              // eV
writeParticles yes;

```

Output in `ionTracker/`, for each patch: `<patch>_energy.dat`, `<patch>_angle.dat`, `<patch>_energyAngle.dat` (the joint distribution) and, with `writeParticles`, `<patch>_particles.dat` with the energy [eV], the angle to the normal [deg] and the RF phase of arrival of every ion. `summary.dat` has the flux, mean energy and mean angle at each patch.

```

import numpy as np, matplotlib.pyplot as plt
e, f = np.loadtxt("ionTracker/electrode_energy.dat", unpack=True)
a, g = np.loadtxt("ionTracker/electrode_angle.dat", unpack=True)
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].semilogy(e, f); ax[0].set_xlabel("energy (eV)")
ax[1].plot(a, g);    ax[1].set_xlabel("angle to the normal (deg)")
plt.show()

```

- **energyFrame** must be that of the cross-section file. The Phelps ion data on LXCat are against the centre-of-mass energy. For Ar^+ in Ar, **crossSections** `PhelpsAr`; needs no file.
- **maxCollisionEnergy** must be above the largest ion energy (about the largest sheath voltage). The log reports “tests above the null-collision frequency”; it must be 0.
- **launch** `sheath` follows the ions only through a layer at the patches: the sheath and five mean free paths by default, or **launchDistance**. The log prints the thickness that was used. The flux in `summary.dat` should equal the ion flux of the fluid solution at the wall.
- Each ion is launched at a phase of the period drawn in proportion to the ions produced in its cell, or to the flux into the layer through its face, at that phase (**uniformLaunchTime** `yes`; launches uniformly in time). **period** and **nPhases** need not be those of the RF period: for a pulsed discharge give the pulse period and the outputs of one pulse; the phase of arrival is then the time within the pulse.

6 Other conditions: what to change

To change	Edit
Pressure	0/p [Pa] and 0/<gas> [$1/\text{m}^3$], consistent with $p = Nk_B T$. No new tables. The time to the steady state and the sheath thickness change: revise the run length and the mesh.
Gas temperature	0/T, 0/Tion, 0/<gas>, gasTemperature in <code>boltzmannDict</code> ; new electron tables.
Voltage	amplitude in 0/Phi. Raise maxCollisionEnergy with it. For a given power or other waveforms use the <code>rfPowerElectrode</code> condition (main manual).

Frequency	<code>frequency</code> in <code>0/Phi</code> ; <code>deltaT</code> , <code>deltaTMax</code> and <code>deltaTMin</code> (a whole number of steps per period), <code>endTime</code> and the output interval in <code>controlDict</code> ; <code>period</code> in <code>ionTrackerDict</code> and the period in <code>Allrun</code> .
Gap	<code>convertToMeters</code> and the number of cells in <code>blockMeshDict</code> ; <code>L</code> in <code>setExpressionFieldsDict</code> .
Secondary emission	<code>seec</code> in <code>0/electron</code> and <code>0/Te</code> .
Gas	New cross-section files; species in <code>reactions</code> , <code>thermoProperties</code> , <code>plasmaProperties</code> (<code>backgroundGas</code> and the species entries) and the field files in <code>0/</code> ; reactions with their energy losses in <code>J</code> ; <code>gases</code> and <code>tables</code> in <code>boltzmannDict</code> ; the ion mobility table; <code>ion</code> , <code>ionMass</code> , <code>gasDensity</code> and the cross sections in <code>ionTrackerDict</code> ; the field names in <code>controlDict</code> (<code>objectNames</code> , <code>fieldAverage</code>) and <code>fvSolution</code> .
Gas mixture	<code>gases ((Ar 0.9) (He 0.1))</code> ; in <code>boltzmannDict</code> with the cross sections of both in the file. <code>ionTracker</code> follows one ion species in one background gas.

Ion mobility table. `constant/mu_<ion>` holds E/N [Td] and μN . Take it from measurements, or from the ion-neutral cross sections: the table of the example was obtained by running `ionTracker` in a uniform field and reading the drift velocity from `ionTrackerVelocity` (reduced mobility $10.2 \text{ cm}^2/\text{Vs}$ at 10 Td).

7 Checks

- **Steady state:** the period averages no longer change between outputs.
- **Time step:** the example uses 250 steps per period. Repeat the last periods with a smaller step and compare the density (this has not been done for the example; in the 1 Torr argon example at 40 MHz a five times larger step changed the peak density by 8 %).
- **Mesh:** several cells across the sheath; with `fluxScheme scharfetterGummel` the 1 Torr argon example is converged to about 1 % with 200 cells.
- **Number of ions:** about 10^5 per electrode for a smooth distribution; the statistical error of a histogram bin is the inverse square root of the number of ions in it.
- **Tracked layer:** repeat with a larger `launchDistance`; the distributions should not change. In argon at 1 Torr, 1.6 mm and 3 mm agree within 2 %.
- **Threads:** the scored ions are the same for any `nThreads`.

8 Results of the example and comparison with a particle-in-cell simulation

Fluid solution. The published benchmark values are from five particle-in-cell codes that agree with each other to 0.5 %.

	fluid (<code>somaFoam</code>)	benchmark	difference
Ion density at the mid-plane [10^{15} m^{-3}]	1.70	2.57	−34 %
Electron temperature at the mid-plane [eV]	4.34	3.65	+19 %
Ion current at each electrode [A/m^2]	0.268	0.186	+44 %

The fluid model, with rate and transport coefficients against the local electron temperature, is far from the kinetic solution of the bulk plasma at this pressure.

Particle-in-cell run. `warpX/inputs` runs the same case with WarpX (version 26.10), with the same cross sections: 512 cells and 3200 steps per period as in the benchmark, but 20 particles per cell of variable weight, a cosine initial density and 2500 periods, three runs with different seeds. The ion current at the electrodes, 0.185 A/m^2 , agrees with the benchmark and is steady from period 250. The mid-plane density (3.2 and still rising at the end) and electron temperature (2.7 eV) do not: this run is not a reference for the bulk plasma, probably for lack of particles. Its ions at the electrodes (65 000, both electrodes, after period 1400) are

compared below with **ionTracker** run on the field of the particle-in-cell run (8 phases of the period, average of the last 32 periods) and on the field of the fluid solution.

At the electrodes	PIC-MCC (WarpX)	ionTracker , PIC field	ionTracker , fluid field
Mean energy [eV]	3.07	2.98	3.20
Median energy [eV]	1.92	1.87	2.05
90 % / 99 % of the ions below [eV]	7.5 / 15.2	7.2 / 14.7	7.7 / 15.9
Mean angle to the normal [deg]	11.7	11.7	11.5
Median angle [deg]	7.7	7.8	7.6
Within 10° of the normal	60.4 %	59.7 %	61.1 %
Beyond 30°	8.1 %	8.1 %	7.8 %
Difference of the histograms from PIC-MCC, energy / angle	–	1.1 % / 1.0 %	2.4 % / 1.3 %

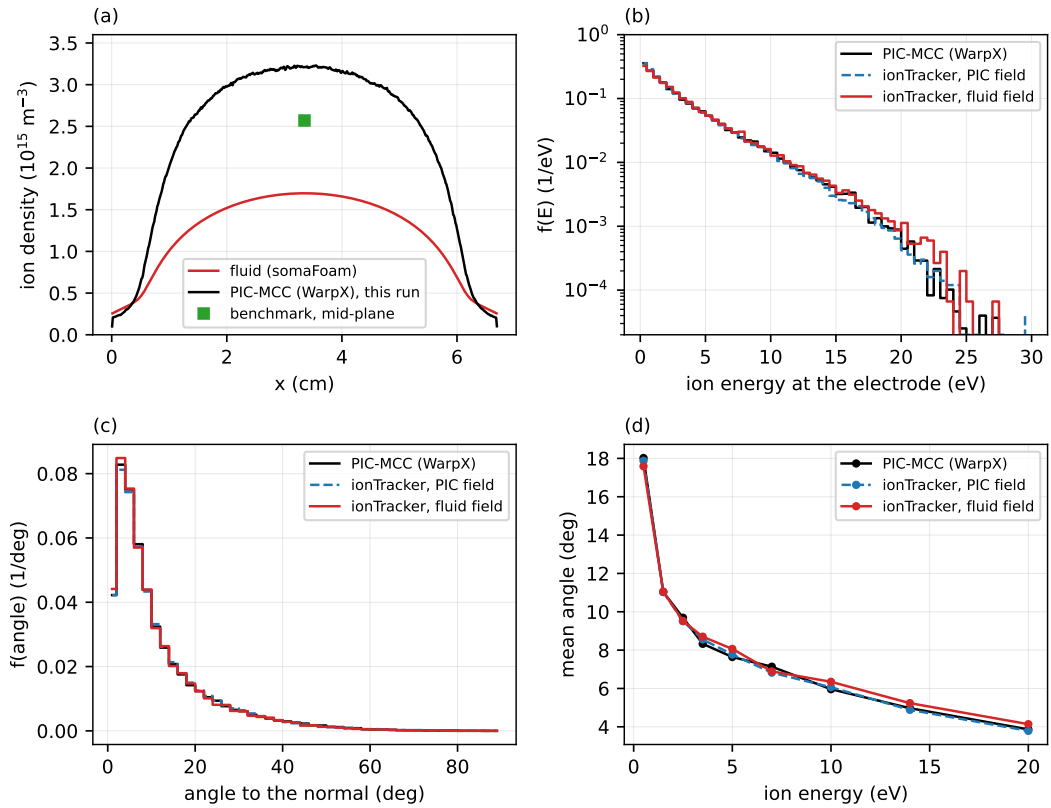


Figure 1: Example case. (a) Period-averaged ion density. (b) Energy and (c) angular distributions of the He^+ ions at the electrodes. (d) Mean angle against energy.

- On the same field, **ionTracker** and the particle-in-cell code agree within the statistical error of the histograms (about 1.4 %).
- On the fluid field the energies are about 4 % higher (the period-averaged field at the wall is 22.0 kV/m against 20.1 kV/m), and the angles agree, although the density and the ion current of the fluid solution are far off: in this collisional sheath the distributions are set by the sheath field and the mean free path.
- The ion flux, and with it the absolute scale of the distributions, is that of the fluid solution: 44 % too high here.
- Both codes use the same model of ion–neutral scattering: the comparison does not test it.
- For the run on the particle-in-cell field the profile of the ion source was estimated from the electrons at two phases of the period.

Cost. Step 1: 8 s (3 threads). Step 2: about 6.5 h on one core (1.5 million time steps of 15 ms, 256 cells). Step 3: 100 000 ions in 6 min on one thread; the tracked layer was 7 mm thick. The particle-in-cell runs took 16 h each.

9 What has been verified, and limits

- **electronBoltzmann:** analytic distributions (constant cross section, constant collision frequency); the Monte Carlo method reproduces the Reid ramp benchmark.
- **ionTracker:** collisionless RF sheath against an independent integration; energy and angle with isotropic scattering in a uniform field against an independent Monte Carlo calculation; the Davis–Vanderslice distribution of a collisional sheath; ion mobilities in argon and helium.
- **ionTracker** against a particle-in-cell simulation of the example case, for the energy and angular distributions in a collisional RF sheath (section 8). The particle-in-cell run did not reproduce the bulk plasma of the benchmark; a comparison of the whole chain with a converged run is not available.
- The ions follow the field of the fluid solution: an error of the fluid model in the sheath voltage or thickness goes into the distributions, and the ion flux is that of the fluid solution. Rates and transport against the local electron temperature are an approximation at low pressure, where the electrons are not in equilibrium with the local field.
- Ion–neutral scattering is the sum of an isotropic and a backward (charge exchange) part. This reproduces ion transport; the wide-angle part of the angular distribution depends on this model.
- The two-term solution treats ionisation as an energy loss and has no electron–electron collisions.
- **ionTracker** runs on one node (threads), on a static mesh, for one ion species; fast neutrals are not followed. The sheath launch has been run in one dimension only.